

单基地被动 UAV MDS 代码详细解释版说明书

面向后续改代码、查错误、写组会补充材料的源码级阅读手册。本文聚焦

`rt_monostatic_passive_uav_mds.py` 和 `validate_monostatic_passive_uav_mds.py` 的函数职责、数据流、关键变量和验证逻辑。

逐 snapshot

Sionna RT PathSolver

单基地被动反射

多散射点叶片

STFT / MDS

1. 这份代码手册怎么读

这份说明书不是重新讲一遍物理背景，而是把代码按执行顺序拆开。推荐阅读顺序是：先看主数据流，再看配置和几何，再看 Sionna RT 调用，最后看 STFT、输出和验证脚本。

命令行参数

- > `PassiveUavMdsConfig`
- > `scatterer_trajectories()` 预生成所有 snapshot 几何
- > `setup_scene()` 创建一个 BS TX + 多个 probe RX
- > for snapshot:
 - `update_probe_positions()`
 - `run_solver()`
 - `one_way_channels_by_rx()`
 - `monostatic_return() = sum_k weight_k * h_k(t)^2`
- > `add_awgn()`
- > `compute_spectrogram()`
- > `save_outputs()` 保存 npz / png / summary

核心思想一句话：UAV 机身和叶片散射点不是主动发射机，而是移动的被动 probe receiver；Sionna RT 每帧只求 BS 到这些 probe 的一程复信道，再用 $h_k(t)^2$ 构造近似单基地往返回波。

2. 主程序函数索引

源码位置	函数 / 类	作用	后续修改时最常碰到什么
44	<code>PassiveUavMdsConfig</code>	集中管理仿真、几何、STFT、输出路径参数。	改载频、采样率、旋翼半径、转速、权重、窗口长度。
124	<code>parse_args()</code>	定义 CLI 参数。	给新实验增加命令行参数。
148	<code>config_from_args()</code>	把命令行参数转为配置对象。	保证 CLI 参数真的写入 config。
175	<code>rotor_offsets()</code>	定义四旋翼中心相对机身的位置。	改四旋翼结构、机臂长度或改成单旋翼。
190	<code>body_position()</code>	计算机身随时间平动后的位置。	换成曲线飞行、加速度或姿态变化。
194	<code>scatterer_state()</code>	计算某一时刻所有散射点的位置、速度、权重和理论 Doppler。	这是最重要的几何建模入口。
254	<code>scatterer_trajectories()</code>	把每个 snapshot 的几何堆叠成数组。	验证几何、保存轨迹、画轨迹图。
273	<code>theory_metrics()</code>	从理论瞬时频率估计 body Doppler 与 micro-Doppler 展宽。	改验收标准或 Doppler 理论公式。
292	<code>load_rt_scene()</code>	加载 Sionna 场景并设置频率、天线阵列。	换 Blender 导入场景或天线配置。
318	<code>setup_scene()</code>	添加唯一 BS transmitter 和所有 probe receiver。	确认 UAV 没有被错误建成主动 TX。
343	<code>update_probe_positions()</code>	逐帧更新 probe receiver 的位置。	这是逐 snapshot 动态仿真的关键动作。
348	<code>run_solver()</code>	调用 <code>PathSolver</code> 并取 CIR 系数。	调 max_depth、多径、反射/绕射开关。
368	<code>one_way_channels_by_rx()</code>	把 Sionna CIR 的路径维度相干求和，得到每个散射点一程信道。	最容易因 CIR 张量维度理解错误而出错。
377	<code>monostatic_return()</code>	用 <code>sum weight*h*h</code> 构造单基地复回波。	改散射点 RCS、权重、相位模型。
392	<code>compute_spectrogram()</code>	对复回波做双边 STFT 并归一化成 dB。	调窗长、重叠率、频率分辨率。
432	<code>run_simulation()</code>	主仿真循环，串起所有步骤。	调性能、加缓存、调 verbose、插入新输出。
501	<code>save_outputs()</code>	保存 npz、图和 summary。	增加数据字段或改变输出命名。

源码位置	函数 / 类	作用	后续修改时最常碰到什么
530	<code>plot_results()</code>	生成四联图。	调整 MDS 图、轨迹图、诊断图。
611	<code>write_summary()</code>	写日志指标。	组会表格、验收指标主要来自这里。
669	<code>main()</code>	CLI 入口。	默认运行路径。

3. 配置类：所有参数的入口

`PassiveUavMdsConfig` 是主程序的参数中心。它不仅保存用户输入，也通过 `@property` 计算派生量，例如波长、时间步长、时间轴、Nyquist 频率、机身速度向量、散射点数量和 STFT 频率分辨率。

```

cfg.wavelength = c / fc
cfg.dt          = 1 / fs
cfg.t_vec       = [0, dt, 2dt, ...]
cfg.nyquist     = fs / 2
cfg.body_velocity = body_speed * body_direction / ||body_direction||
cfg.n_scatterers = 1 + n_rotors*n_blades_per_rotor*points_per_blade
cfg.stft_df      = fs / stft_win

```

默认配置是为了可跑通和可验证而缩放过的：`fc=28e9`、`fs=2000`、`snapshots=1024`、`blade_radius=0.02`、`rotation_freq=20`、`points_per_blade=3`、`max_depth=0`。这组参数让微多普勒峰值落在 Nyquist 范围内，同时运行时间不至于太长。

如果后续把叶片半径、转速或载频改大，必须重新检查 `body Doppler ± micro-Doppler peak` 是否超过 `fs/2`。超过以后 STFT 会发生 Doppler 混叠，MDS 看起来会折回到错误频率。

4. CLI：命令行参数如何变成配置

`parse_args()` 只负责声明参数名、类型和默认值。`config_from_args()` 负责把这些参数写入 `PassiveUavMdsConfig`。如果新增参数，只改 `parse_args()` 还不够，也要在 `config_from_args()` 里传进去。

```

python monostatic_passive_uav_mds/rt_monostatic_passive_uav_mds.py \
  --snapshots 1024 \
  --fs 2000 \
  --fc 28e9 \
  --max-depth 0 \
  --body-speed 1.0 \
  --blade-radius 0.02 \
  --rotation-freq 20 \
  --points-per-blade 3

```

`--no-noise` 是一个开关参数。命令行出现它时，`args.no_noise=True`，最终配置里 `add_noise=False`。做理论验证和相位连续性测试时，建议使用无噪声。

5. 几何建模：机身平动 + 叶片相对旋转

5.1 机身轨迹

`body_position(cfg, t)` 实现的是匀速直线飞行：

```
body(t) = body_position0 + body_velocity * t
```

当前默认 `body_position0=(0,0,3)`，`body_velocity=(1,0,0)`，因此 UAV 机身沿 x 方向前进。如果要换成更真实的轨迹，比如圆弧、变速或高度变化，优先从这个函数改。

5.2 旋翼中心

`rotor_offsets()` 返回四个旋翼中心相对机身的固定偏移：

```
[ arm,  arm, 0]
[-arm,  arm, 0]
[-arm, -arm, 0]
[ arm, -arm, 0]
```

每一帧的旋翼中心由 `rotor_center(t)=body(t)+rotor_offset` 得到。这正好对应“机身负责整体平动，旋翼跟随机身移动”的设置。

5.3 叶片多散射点

`scatterer_state()` 是几何建模的核心函数。它先把机身作为第 0 个散射体，再为每个 rotor、每片 blade、每个半径采样点生成一个叶片散射点。

```
omega = 2*pi*rotation_freq
angle = omega*t + rotor_phase + blade_phase
rel    = [rho*cos(angle), rho*sin(angle), 0]
pos    = body(t) + rotor_offset + rel
v_rot  = [-omega*rho*sin(angle), omega*rho*cos(angle), 0]
vel    = body_velocity + v_rot
```

默认四旋翼、每个旋翼两片叶片、每片叶片三个散射点，因此叶片散射点数是 `4*2*3=24`；加上机身散射点，总散射体数为 25。`points_per_blade` 越大，叶片半径方向采样越密，STFT 上的微多普勒能量越容易形成连续包络。

5.4 理论瞬时 Doppler

函数中还顺手计算每个散射点相对 BS 的理论单基地 Doppler：

```
los_unit = (scatterer_position - bs_position) / range
range_rate = dot(scatterer_velocity, los_unit)
f_mono = -2 * range_rate / wavelength
```

这里的系数 2 来自单基地往返路径。机身速度贡献 standard Doppler；叶片旋转速度贡献 micro-Doppler。二者不是两条互不相关的信号，而是在每个散射点的瞬时径向速度里相加，最后体现在复回波相位的时间变化上。

6. Sionna RT 场景：BS 是唯一主动 TX

6.1 加载场景和天线阵列

`load_rt_scene()` 支持两个内置场景：`floor_wall` 和 `etoile`。加载后设置 `scene.frequency=cfg.fc`，并把 TX/RX 阵列都设为单天线、各向同性、V 极化：

```
scene.tx_array = PlanarArray(1, 1, pattern="iso", polarization="V")
scene.rx_array = PlanarArray(1, 1, pattern="iso", polarization="V")
```

当前版本先用单天线简化验证相位和 MDS。后续如果要模拟阵列雷达，可以先扩展 `PlanarArray`，再检查 `paths.cir()` 输出维度的读取方式。

6.2 创建 TX 和 probe RX

`setup_scene()` 只添加一个名为 `bs_radar` 的 `Transmitter`。机身和叶片点全部添加为 `Receiver`，名字类似 `probe_00_body`、`probe_01_rotor0_blade0_rho0.0067`。

```
scene.add(Transmitter(name="bs_radar", position=bs_position))
scene.add(Receiver(name=f"probe_{idx:02d}_{label}", position=scatterer_pos))
```

这一步保证“UAV 是被动反射体”的建模约束：场景里真正主动发射的只有 BS。probe receiver 只是为了让 Sionna RT 给出 BS 到该散射点位置的一程复信道。

6.3 逐帧更新 probe 位置

`update_probe_positions()` 每个 snapshot 都把所有 probe receiver 移动到对应位置。这一句是“逐 snapshot 动态更新”的工程落点：

```
scene.get(name).position = pos.tolist()
```

因为 Sionna RT 本身不是连续时间运动仿真器，所以这里采用离散 snapshot：每一帧更新几何、重新调用 `PathSolver`，最终由连续复相位序列产生 Doppler。

7. PathSolver 和 CIR：从射线路径到一程复信道

`run_solver()` 调用 Sionna RT 的 `PathSolver`。默认 `max_depth=0`，只开 LOS；反射、多径可通过 `max_depth` 和相关布尔开关继续扩展。

```
paths = solver(scene,
    max_depth=cfg.max_depth,
    los=True,
    specular_reflection=True,
    diffuse_reflection=False,
```

```

diffraction=False,
synthetic_array=False)

a, tau = paths.cir(
    sampling_frequency=cfg.cir_sampling_frequency,
    normalize_delays=False,
    out_type="numpy")

```

代码只取 `a`，也就是每条路径的复系数。随后 `one_way_channels_by_rx()` 按 receiver 抽取并对路径维度相干求和：

```
h_k(t) = sum_p a[k, 0, 0, 0, p, 0]
```

这一步的含义是：对于第 k 个散射点，把 BS 到这个 probe 的所有路径复幅度加起来，得到该散射点的一程复信道。如果 `max_depth=0`，通常只有 LOS 路径；如果打开反射，`sum_p` 会把多径也加进来。

最容易出错的是 CIR 张量维度。当前代码假设单 TX、单 TX 天线、单 RX 天线和单时间抽头，所以读取 `a[:n_receivers, 0, 0, 0, :, 0]`。如果改成阵列、多 TX 或更复杂的链路，这里必须重新检查维度。

8. 单基地被动回波：为什么是 $h_k(t)^2$

`monostatic_return()` 是雷达回波合成的核心近似：

```
s(t) = sum_k weight_k * h_k(t) * h_k(t)
```

其中 `h_k(t)` 是 BS 到第 k 个散射点的一程复信道。对单基地雷达，发射和接收在同一点，往返路径可以用互易性近似为同一条一程路径走两次，所以幅度损耗近似平方，相位也变成往返相位。

`weight_k` 是散射点的简化反射强度。当前机身权重默认为 `1.0`，叶片总权重默认为 `0.18`，并在每片叶片的多个半径点之间平均分配。这不是严格 RCS，而是点散射体级的工程近似。

严格电磁散射需要目标表面网格、材料、极化、入射角相关 RCS 和遮挡关系。当前 `h_k(t)^2` 模型适合验证 MDS 结构、相位连续性和 Doppler 频率位置；不应直接声称等价于完整电磁散射求解。

9. 主循环 `run_simulation()`

`run_simulation()` 是主程序最重要的执行函数。它先检查参数合法性，创建随机数、场景和 solver，然后预生成轨迹数组。循环中每一帧做四件事：

1. 把所有 probe receiver 移到当前 snapshot 的位置。
2. 调用 `PathSolver` 得到当前几何下的 CIR。
3. 把 CIR 相干求和为每个散射点的一程信道 `h_k(t)`。
4. 用 `sum weight*h*h` 生成该帧的单基地复回波。

```

for idx in range(cfg.snapshots):
    update_probe_positions(scene, probe_names, positions[idx])
    a = run_solver(scene, solver, cfg)
    h = one_way_channels_by_rx(a, cfg.n_scatterers)
    one_way_h[idx] = h
    signal_clean[idx] = monostatic_return(h, weights)

```

循环结束后，根据 `add_noise` 选择是否加复高斯白噪声，再计算 STFT、理论指标和频谱轮廓指标。返回值是一个字典，后续保存、绘图、验证都从这个字典里取数据。

10. STFT：从复回波到 MDS

`compute_spectrogram()` 对复信号做双边谱 STFT。它使用 Hann 窗，窗口长度为 `stft_win`，重叠率默认为 0.75。

```

z = spectrogram(sig, fs=cfg.fs,
                window=hann(n_win),
                nperseg=n_win,
                noverlap=n_overlap,
                mode="complex",
                return_onesided=False)

f_centered = fftshift(fftfreqz(n_win, 1/fs))
S_dB = 20*log10(abs(fftshift(z)) + 1e-12)
S_dB = S_dB - max(S_dB)

```

这里使用 `return_onesided=False` 是因为输入是复基带信号，正负 Doppler 都有物理意义。`fftshift` 把 0 Hz 放到频率轴中间，便于观察 body Doppler 和 micro-Doppler 展宽。

参数	代码含义	实际影响
<code>fs</code>	snapshot rate	决定 Doppler Nyquist 范围，即 $\pm fs/2$ 。
<code>stft_win</code>	每个 STFT 窗的样本数	频率分辨率 $df=fs/stft_win$ ，窗口越长频率越细但时间越粗。
<code>stft_overlap_ratio</code>	相邻窗重叠率	提高时间轴平滑度，但不改变基本频率分辨率。
<code>S_dB -= max(S_dB)</code>	归一化幅度	最大值为 0 dB，便于比较不同实验图。

11. 频谱指标：主峰和展宽范围

`frequency_profile_metrics()` 先对 STFT 沿时间方向取最大值，得到 Doppler profile：

```

profile(f) = max_t S_dB(f, t)

```

然后取 profile 最大值对应的频率作为 `STFT peak`，并用 `-35 dB` 阈值估计 MDS 支撑范围。当前默认日志中：

理论 body Doppler	-186.796 Hz
理论 micro-Doppler 峰值	469.468 Hz
理论展宽范围	[-656.264, 282.672] Hz
STFT 主峰	-187.500 Hz
STFT 支撑范围	[-671.875, 296.875] Hz @ -35 dB
STFT 频率 bin	7.8125 Hz

主峰与理论 body Doppler 的误差小于一个频率 bin，因此满足默认验收标准。支撑范围略超理论边界，主要来自 STFT 窗函数泄漏、噪声、阈值选择和离散频率 bin。

12. 保存输出：npz、图和 summary

12.1 npz 数据

`save_outputs()` 使用 `np.savez_compressed()` 保存完整结果。重要字段包括：

字段	含义
<code>signal</code>	最终用于 STFT 的复回波，可能包含噪声。
<code>signal_clean</code>	无噪声单基地复回波，用于相位连续性诊断。
<code>one_way_h</code>	每个 snapshot、每个散射点的一程复信道。
<code>f_stft</code> , <code>t_stft</code> , <code>S_dB</code>	MDS 的频率轴、时间轴和 dB 幅度矩阵。
<code>positions</code> , <code>velocities</code>	所有散射点的轨迹和速度。
<code>rotor_centers</code>	旋翼中心轨迹，用于检查跟随机身平动。
<code>monostatic_freq_theory</code>	每个散射点的理论瞬时单基地 Doppler。
<code>theory_json</code> , <code>profile_json</code> , <code>config_json</code>	理论指标、STFT 指标和配置快照。

12.2 四联图

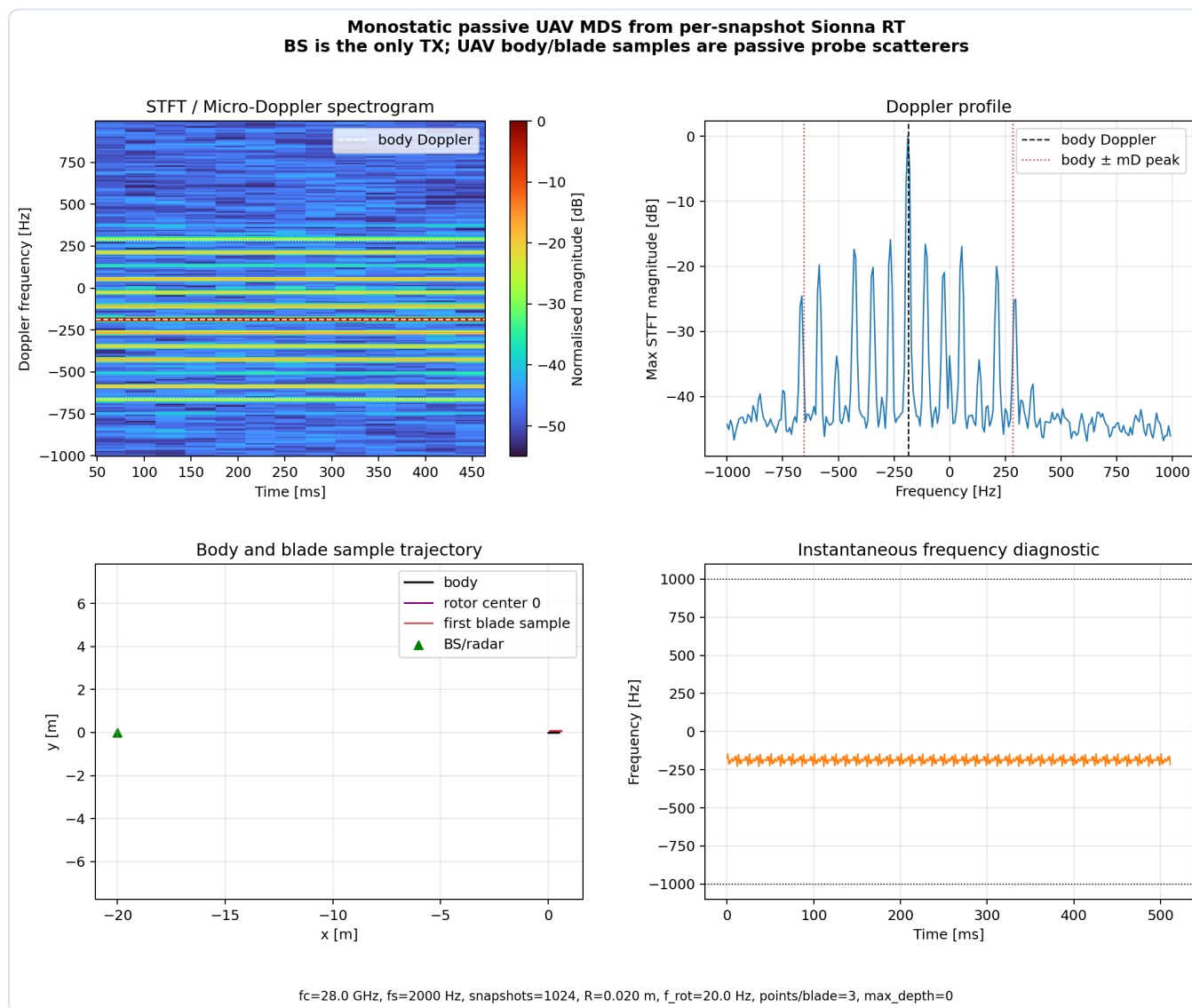


图 1：主程序生成的 MDS、Doppler profile、轨迹和瞬时频率诊断图。

`plot_results()` 生成四个子图。左上为 STFT/MDS，白色虚线标理论 body Doppler，白色点线标理论 `body  $\pm$  mD peak`。右上为 Doppler profile，用于读主峰和展宽范围。左下是机身、旋翼中心、首个叶片点和 BS 的平面轨迹。右下是无噪声复回波的瞬时频率诊断，用来发现相位跳变或 Doppler 混叠。

12.3 summary 文本

`write_summary()` 把关键参数和验收指标写入 `outputs/logs/monostatic_passive_uav_mds_summary.txt`。组会汇报中的参数表和结果表可以直接引用这里的数据。

13. 入口函数 main()

`main()` 的执行顺序很短，但它串起了整个 CLI 流程：

```
args = parse_args()
cfg = config_from_args(args)
ensure_output_dirs()
print_header(cfg)
```

```
result = run_simulation(cfg, verbose=True)
save_outputs(result)
```

因此默认运行命令就是：

```
python monostatic_passive_uav_mds/rt_monostatic_passive_uav_mds.py
```

14. 验证脚本结构

`validate_monostatic_passive_uav_mds.py` 不是简单跑一遍程序，而是把几何、STFT、Doppler、相位和消融实验拆成多个测试组。每个测试组失败后会写入 `outputs/logs/validation_report.txt`。

源码位置	函数	验证内容
30	<code>check()</code>	统一断言函数，失败时抛出 <code>ValidationFailure</code> 。
35	<code>validation_cfg()</code>	构造快速测试配置：128 snapshots、无噪声、 <code>max_depth=0</code> 。
47	<code>test_geometry()</code>	检查机身、旋翼中心、叶片半径和整体平移。
77	<code>test_stft_output()</code>	检查 STFT 频率轴、矩阵 shape、有限性和 dB 归一化。
90	<code>test_default_rt_signal_and_doppler()</code>	跑快速 RT 仿真，检查主峰与理论 body Doppler 一致，展宽范围合理。
117	<code>test_phase_continuity()</code>	检查无噪声信号 unwrap 相位没有异常跳变，瞬时频率不越 Nyquist。
136	<code>test_ablations()</code>	执行四个消融实验：无旋转、无机身平动、关闭叶片、关闭机身。
198	<code>main()</code>	逐项运行测试并写 validation report。

14.1 几何测试

`test_geometry()` 的目的不是看图，而是用数组直接证明几何关系正确：

```
body(t) == body0 + V*t
rotor_center(t) - body(t) == constant offset
||blade_point(t) - rotor_center(t)|| == rho
rotor_center(final) - rotor_center(initial) == body shift
```

这组测试能防止最常见的错误：叶片自己转了，但没有跟随机身平移；或者旋翼中心跟了，叶片点的半径却被写错。

14.2 STFT 输出测试

`test_stft_output()` 构造一个 125 Hz 的理想复指数，不依赖 Sionna RT。它只检查 STFT 函数本身是否输出正确格式：

```
len(f_stft) == stft_win
f_stft 单调递增
f_stft 中心 bin 是 0 Hz
S_dB.shape == (len(f_stft), len(t_stft))
```

```
max(S_dB) == 0 dB
df == fs/stft_win
```

14.3 Doppler 一致性测试

`test_default_rt_signal_and_doppler()` 使用快速 RT 配置运行一次仿真。验收标准有两个：

- 1. STFT 主峰频率接近理论 body Doppler，误差不超过一个 STFT bin。
- 2. MDS 的主要展宽不明显跑出 `body Doppler ± theoretical micro-Doppler peak`。

第二条允许 `8*df` 的容差，是为了容纳 STFT 泄漏、有限时间窗和离散频率 bin。

14.4 相位连续性测试

`test_phase_continuity()` 使用无噪声 `signal_clean`，先做 `unwrap(angle(signal_clean))`，再检查相邻相位差和瞬时频率：

```
dphi = diff(unwrap(angle(signal_clean)))
inst_freq = gradient(phase, 1/fs)/(2*pi)
abs(dphi) < 0.99*pi
max(abs(inst_freq)) < 0.99*nyquist
```

如果这个测试失败，通常意味着采样率不够、相位出现非物理跳变，或者某些 snapshot 的信号幅度接近零导致相位不稳定。

14.5 四个消融实验

Case	参数	理论预期	测试关注点
A	<code>rotation_freq=0</code>	无旋转，不应有 micro-Doppler 展宽。	支撑宽度应较窄。
B	<code>body_speed=0</code>	无机身平动，能量应围绕 0 Hz 展宽。	主能量质心接近 0。
C	<code>blade_weight=0</code>	关闭叶片，只剩机身 Doppler 水平线。	谱宽应较窄。
D	<code>body_weight=0</code>	关闭机身，旋翼 micro-Doppler 更突出。	body Doppler bin 能量减弱，谱宽变宽。

当前验证报告结果是：

```
PASS geometry
PASS stft_output
PASS default_rt_signal_and_doppler
PASS phase_continuity
PASS ablations
ALL TESTS PASSED
```

15. 常见改动位置

想做的事	主要改哪里	需要同步检查什么
提高 micro-Doppler 精度	<code>points_per_blade</code> 、 <code>blade_radius</code> 、 <code>rotation_freq</code>	采样率是否足够，运行时间是否可接受。
改成单旋翼或不同旋翼布局	<code>n_rotors</code> 、 <code>rotor_offsets()</code> 、循环相位	验证脚本当前写死四旋翼，需要一起改。
加入真实多径	<code>run_solver()</code> 中 <code>max_depth</code> 和反射/绕射开关	运行时间、路径数量、相干求和和相位连续性。
换 Blender 导入场景	<code>load_rt_scene()</code>	材料、坐标系、BS/UAV 高度、单位尺度。
加入 RCS 或角度相关散射	<code>weights</code> 生成逻辑和 <code>monostatic_return()</code>	权重是否随入射角、极化、频率和遮挡变化。
改 STFT 清晰度	<code>stft_win</code> 、 <code>stft_overlap_ratio</code>	df、时间分辨率、主峰误差验收阈值。
加速仿真	<code>snapshots</code> 、 <code>max_depth</code> 、散射点数量	MDS 分辨率和统计稳定性。

16. 最容易踩坑的代码点

- `points_per_blade` 增加后，散射点数量线性增加，每个 snapshot 的 PathSolver 接收端也会增加，运行时间会上升。
- `fs` 太低会导致 Doppler 混叠；不是 STFT 画错，而是输入相位已经采样不够。
- `stft_win` 太短会让主峰落点粗糙；太长会让时间变化被平均掉。
- 改阵列或多 TX 后，`one_way_channels_by_rx()` 的张量索引必须重写。
- 把 UAV 散射点误建成 transmitter 会破坏“BS 唯一 TX、UAV 被动反射”的模型假设。
- `$h_k(t)^2$` 是点散射体单基地近似，不是完整网格电磁散射。
- 打开多径后，STFT 展宽可能包含环境路径相干变化，不再是纯叶片 micro-Doppler。
- 如果信号幅度在某些时刻接近零，相位 unwrap 和瞬时频率诊断可能出现尖峰。

17. 运行命令速查

```
# 默认运行，生成图、npz 和 summary
python monostatic_passive_uav_mds/rt_monostatic_passive_uav_mds.py

# 快速烟测
python monostatic_passive_uav_mds/rt_monostatic_passive_uav_mds.py \
    --snapshots 64 --fs 2000 --stft-win 32 --no-noise --max-depth 0
```

```
# 自动验证
python monostatic_passive_uav_mds/validate_monostatic_passive_uav_mds.py

# Case A: 无旋转
python monostatic_passive_uav_mds/rt_monostatic_passive_uav_mds.py \
    --rotation-freq 0 --no-noise

# Case B: 无机身平动
python monostatic_passive_uav_mds/rt_monostatic_passive_uav_mds.py \
    --body-speed 0 --no-noise

# Case C: 关闭叶片散射
python monostatic_passive_uav_mds/rt_monostatic_passive_uav_mds.py \
    --blade-weight 0 --no-noise

# Case D: 关闭机身散射
python monostatic_passive_uav_mds/rt_monostatic_passive_uav_mds.py \
    --body-weight 0 --no-noise
```

18. 这版代码的严谨表述

如果写进组会或论文方法章节，建议表述为：

本实验采用逐 snapshot 的 Sionna RT 几何更新方式，将 BS/Radar 建模为唯一主动发射端，将 UAV 机身和叶片多散射点建模为被动 probe receiver。每个 snapshot 中，先根据机身平动和叶片相对旋转更新所有散射点绝对位置，再调用 PathSolver 获得 BS 到散射点的一程复 CIR。在单基地互易近似下，散射点的往返回波由一程复信道平方构造，即 $s_k(t) = w_k h_k(t)^2$ ，总回波为所有散射点相干叠加。该方法能够验证 standard Doppler 与 micro-Doppler 在复相位和 STFT/MDS 上的叠加规律，但当前版本仍属于点散射体级近似，尚未包含完整叶片网格、真实 RCS、角度相关材料散射和复杂遮挡效应。

生成文件：[monostatic_passive_uav_mds_code_manual.html](#) 与 [monostatic_passive_uav_mds_code_manual.pdf](#)。主要参考源码：[rt_monostatic_passive_uav_mds.py](#)、[validate_monostatic_passive_uav_mds.py](#)。